

Improving UAV control: a comparison between NMPC with observer and with adaptation

Abstract—In recent decades, the use and fields of application of UAVs (unmanned aerial vehicles) have grown exponentially. They are used for civilian (deliveries, entertainment, agriculture, plant inspection, etc.) and military purposes, with increasing success. Despite this, due to their complex dynamics, modeling their behavior, and even more, controlling them, remain open problems. This work aims to compare the performance exhibited in trajectory tracking in presence of different classes of disturbances, when two NMPC (nonlinear model predictive control) extensions (with adaptation or observer) are used for control. The simulations clearly show that there is no absolute winner, but rather that each control technique has its pros and cons. Specifically, the ability to reject a constant additive disturbance was found using an observer alongside the classic NMPC, but it was unable to perform well in the presence of model uncertainties: the behavior of the NMPC with adaptation is the opposite. It can thus be said that the best technique depends on the working conditions.

UAV | disturbances | NMPC | observer | adaptation

Introduction

Controlling UAVs is a fascinating and complex world: while on one hand it is possible to achieve remarkable results and advantages (for example, many shipping companies use these drones to make deliveries to remote locations that would be difficult or much more expensive to reach by other means of transport), on the other hand their non-linear behavior and the high number of significant aerodynamic effects to take into account make them very difficult to control. There are standard and fairly well-established control techniques in the literature that are quite simple to implement, but they are constrained by the use of a very specific drone model (reducing the degrees of freedom of those who design the control law) and, above all, their use alone is not sufficient to guarantee robustness to disturbances or parametric uncertainties. The use of *Model Predictive Control*, on the other hand, although more complex to implement, guarantees complete freedom in the choice of model and control objectives, which can be expressed by choosing any cost function to be optimized. A further advantage of this technique is the ease of imposing constraints on inputs, states, and outputs, so as to reflect operating conditions closer to real ones. Despite all this, even the MPC, given that the predictions on which it is based come from the available model, is not robust to uncertainties or disturbances, so on its own it does not guarantee good performance in terms of tracking, which is why two extensions of this technique will be analyzed. The first is called $\mathcal{L}_1$ -NMPC and consists of calculating a compensation term: this

is added to the inputs calculated according to the nominal model in order to counteract the effect of parametric uncertainties. The second consists of adding an observer, capable of estimating a constant additive disturbance, and making the controller work taking this into account. The robustness of these techniques will be analyzed using both parametric uncertainties and constant disturbances. This is consistent with what occurs in real applications: the parameters of a UAV can change if, for example, it is carrying a load (unknown change in total mass and inertia properties), and a disturbance can occur on a windy day (constant disturbance in the x-y plane). The aerial drone that we consider is a quadrotor and its model and the formulation of the NMPC with adaptation were developed with strong inspiration from the work (1).

Controllers implementation

A. Quadrotor Model. First, two reference frames were defined: World (indicated with W) and Body (indicated with B and placed at the drone's center of mass). The rotors are coplanar in the x-y plane of frame B. We will indicate the vectors expressed in the world frame without a subscript, while those in the body frame will be indicated with the subscript B. The orientation is represented using unit quaternions $\mathbf{q} = (q_w, q_x, q_y, q_z)$, and the product between a vector and a quaternion is indicated with $\odot$, i.e. $\mathbf{q} \odot \mathbf{v} = \mathbf{q} \cdot [0 \ \mathbf{v}^T]^T \cdot \bar{\mathbf{q}}$, while $\bar{\mathbf{q}}$ indicates the conjugate of the quaternion $\mathbf{q}$. The quadrotor is modeled as follows

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}} \\ \dot{\mathbf{q}} \\ \dot{\mathbf{v}} \\ \dot{\boldsymbol{\omega}}_B \end{bmatrix} = \begin{bmatrix} \mathbf{v} \\ \mathbf{q} [0 \ \boldsymbol{\omega}_B^T/2]^T \\ \mathbf{q} \odot \mathbf{T}_B/m + \mathbf{g} \\ \mathbf{J}^{-1} [\boldsymbol{\tau}_B - \boldsymbol{\omega}_B \times \mathbf{J} \boldsymbol{\omega}_B] \end{bmatrix} \quad (1)$$

where $\mathbf{g} = [0, 0, -9.81]^T$ is the gravitational acceleration,

$$\mathbf{T}_B = \begin{bmatrix} 0 \\ 0 \\ \sum T_i \end{bmatrix} \quad (2)$$

is the total thrust produced from the rotors (T_i is the thrust produced by motor i), $\mathbf{J} = \text{diag}(J_x, J_y, J_z)$ is the inertia matrix, m is the quadrotor nominal mass and

$$\boldsymbol{\tau}_B = \mathbf{P} \begin{bmatrix} T_0 \\ T_1 \\ T_2 \\ T_3 \end{bmatrix} \quad (3)$$

is the body torque, where $\mathbf{P}$ is the allocation matrix given by

$$\mathbf{P} = \begin{bmatrix} -d_{x0} & -d_{x1} & d_{x2} & d_{x3} \\ d_{y0} & -d_{y1} & -d_{y2} & d_{y3} \\ -c_\tau & c_\tau & -c_\tau & c_\tau \end{bmatrix} \quad (4)$$

and d_{x_i}, d_{y_i}, c_τ are the distances between the Body frame axis and the rotor i , while c_τ are the rotor drag torque constant as computed using the configuration shown in (1). Since the NMPC works in discrete time, the system is discretized using an explicit Runge-Kutta method of 4th order as explained in (2):

$$\mathbf{x}_{k+1} = f_{RK4}(\mathbf{x}_k, \mathbf{u}_k, \delta t), \quad (5)$$

where

$$\begin{aligned} k_1 &= f(\mathbf{x}_k, \mathbf{u}_k) \\ k_2 &= f(\mathbf{x}_k + \frac{\delta t}{2} k_1, \mathbf{u}_k) \\ k_3 &= f(\mathbf{x}_k + \frac{\delta t}{2} k_2, \mathbf{u}_k) \\ k_4 &= f(\mathbf{x}_k + \delta t k_3, \mathbf{u}_k) \\ \mathbf{x}_{k+1} &= \mathbf{x}_k + \frac{\delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4). \end{aligned} \quad (6)$$

B. NMPC. To implement NMPC, the first thing to do is to define the cost function to take into account the tracking error and control effort and the constraints on states and inputs. The problem will be

$$\begin{aligned} \min_{\mathbf{u}} & (\mathbf{x}_N - \mathbf{x}_N^r)^\top \mathbf{Q} (\mathbf{x}_N - \mathbf{x}_N^r) + \\ & + \sum_{k=0}^{N-1} (\mathbf{x}_k - \mathbf{x}_k^r)^\top \mathbf{Q} (\mathbf{x}_k - \mathbf{x}_k^r) + (\mathbf{u}_k - \mathbf{u}_k^r)^\top \mathbf{R} (\mathbf{u}_k - \mathbf{u}_k^r) \\ \text{s.t.} & : \mathbf{x}_{k+1} = f_{RK4}(\mathbf{x}_k, \mathbf{u}_k, \delta t) \\ & \mathbf{x}_0 = \mathbf{x}_{init} \quad \mathbf{u}_{min} \leq \mathbf{u}_k \leq \mathbf{u}_{max} \end{aligned} \quad (7)$$

The solution is found at each simulation step using the MATLAB *quadprog* command, part of the *optimization toolbox*.

C. $\mathcal{L}_1$ adaptation law. Based on the work (1) we implemented an adaptation law, which will then be added to the outputs provided by the MPC as

$$\mathbf{u}_k = \mathbf{u}_{k,MPC} + \mathbf{u}_{k,\mathcal{L}_1} \quad (8)$$

and $\mathbf{u}_k$ input of the system at instant k .

The calculation of $\mathbf{u}_{k,\mathcal{L}_1}$ is performed as follows

$$\mathbf{u}_{k,\mathcal{L}_1} = \mathbf{u}_{k-1,\mathcal{L}_1} e^{-\omega_{co} T_s} - \hat{\boldsymbol{\sigma}}_{k,m} (1 - e^{-\omega_{co} T_s}) \quad (9)$$

where ω_{co} is the cutoff frequency of the strictly proper continuous time filter $C(s)$, T_s is the sampling time and $\hat{\boldsymbol{\sigma}}(k)$ is the vector of estimated disturbances at k . In particular $\hat{\boldsymbol{\sigma}}(k) = [\hat{\boldsymbol{\sigma}}_m(kT_s) \quad \hat{\boldsymbol{\sigma}}_{um}(kT_s)]^\top$ where the first component is a column vector of dimension 4 of matched disturbances and the second is a column vector of dimension 2 of unmatched disturbances. These estimations can be obtained as

$$\begin{bmatrix} \hat{\boldsymbol{\sigma}}_m(kT_s) \\ \hat{\boldsymbol{\sigma}}_{um}(kT_s) \end{bmatrix} = -\mathbf{I}_6 \mathbf{G}^{-1}(kT_s) \Phi^{-1} \boldsymbol{\mu}(kT_s) \quad (10)$$

where $\mathbf{G}(kT_s) = [\mathbf{g}(\mathbf{R}_B^I), \mathbf{g}^\top(\mathbf{R}_B^I)]$,

$$\mathbf{g}(\mathbf{R}_B^I) = \begin{bmatrix} \frac{e_x^B}{m}, \frac{e_y^B}{m}, \frac{e_z^B}{m}, \frac{e_z^B}{m} \\ \mathbf{J}^{-1} \mathbf{P} \end{bmatrix} \quad (11)$$

$$\mathbf{g}^\perp(\mathbf{R}_B^I) = \begin{bmatrix} \frac{e_x^B}{m}, \frac{e_y^B}{m} \\ \mathbf{0}_{3 \times 1}, \mathbf{0}_{3 \times 1} \end{bmatrix}$$

and e_x^B, e_y^B, e_z^B are the column vectors of the rotation matrix $\mathbf{R}_B^W$ expressing the rotation of the Body frame with respect to the World frame. We defined $\Phi = \mathbf{A}_s^{-1} (e^{(\mathbf{A}_s T_s)} - \mathbf{I})$, with $\mathbf{A}_s$ Hurwitz adaption gain matrix, while $\boldsymbol{\mu}(kT_s) = e^{\mathbf{A}_s T_s} \tilde{\mathbf{z}}(kT_s)$. $\tilde{\mathbf{z}}(kT_s) = \hat{\mathbf{z}}(kT_s) - \mathbf{z}(kT_s)$ and the evolution of the state predicted by the estimator evolves as

$$\hat{\mathbf{z}}_{k+1} = \hat{\mathbf{z}}_k + [\mathbf{f}_k + \mathbf{g}_k(\mathbf{u}_{k,\mathcal{L}_1} + \hat{\boldsymbol{\sigma}}_{k,m}) + \mathbf{g}_k^\perp \hat{\boldsymbol{\sigma}}_{k,um} + \mathbf{A}_s \tilde{\mathbf{z}}_k] T_s. \quad (12)$$

D. Offset-Free-NMPC. To implement this controller, the following estimator is used (3)

$$\begin{bmatrix} \hat{\mathbf{x}}_{k+1} \\ \hat{\mathbf{d}}_{k+1} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_k & \mathbf{B}_d \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \hat{\mathbf{x}}_k \\ \hat{\mathbf{d}}_k \end{bmatrix} + \begin{bmatrix} \mathbf{B}_k \\ \mathbf{0} \end{bmatrix} \mathbf{u}_k - \begin{bmatrix} \mathbf{L}_x \\ \mathbf{L}_d \end{bmatrix} (\mathbf{C} \hat{\mathbf{x}}_k + \mathbf{C}_d \hat{\mathbf{d}}_k - \mathbf{x}_m). \quad (13)$$

where $\hat{\mathbf{x}}_{k+1}$ and $\hat{\mathbf{d}}_{k+1}$ represent the estimate of the state and disturbance for the next step relative to the one considered, $\mathbf{A}_k$ and $\mathbf{B}_k$ are the matrices of the linearized model, $\mathbf{L} = [\mathbf{L}_x \quad \mathbf{L}_d]^\top$ is a gain matrix and $\mathbf{C}_d$ is the constant matrix that accounts for the effect of the disturbance on the output of the system.

Once this is done, the original MPC problem does not change, but simply $\mathbf{u}_k^r$ is replaced by $\bar{\mathbf{u}}_k$ to compensate for the disturbance. The values for this variable are obtained as the solution of the following system:

$$\begin{bmatrix} \mathbf{A}_k - \mathbf{I} & \mathbf{B}_k \\ \mathbf{C} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \bar{\mathbf{x}}_t \\ \bar{\mathbf{u}}_t \end{bmatrix} = \begin{bmatrix} -\mathbf{B}_d \hat{\mathbf{d}}_k \\ \mathbf{r}_k - \mathbf{C}_d \hat{\mathbf{d}}_k \end{bmatrix} \quad (14)$$

where $\mathbf{r}_k$ is the reference at step k .

Simulations

All the simulations use a sample time $T_s = 0.05s$. First of all, for the sake of completeness, we show the desired trajectory and linear velocity profiles, i.e., the outputs of the planner in Fig. 1-2.

Tables 1 to 4 show all the relevant parameters used for the simulations (4).

A. Adaptive MPC results. First, we test the controller's performance in the presence of model uncertainties, using the perturbed parameters instead of the nominal ones.

As can be seen in Fig. 3, initially the error has a highly oscillatory behavior and reaches its maximum magnitude: this is due to the transient of the adaptation term, which vanishes in about two seconds. After this moment, there is an increase in the error on the variables x and y due to the more aggressive

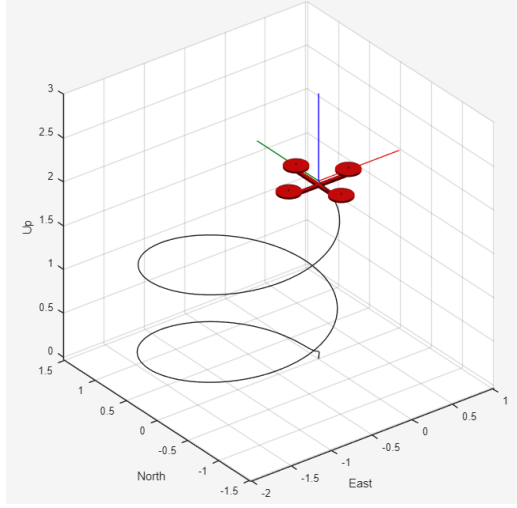


Fig. 1. Desired Trajectory.

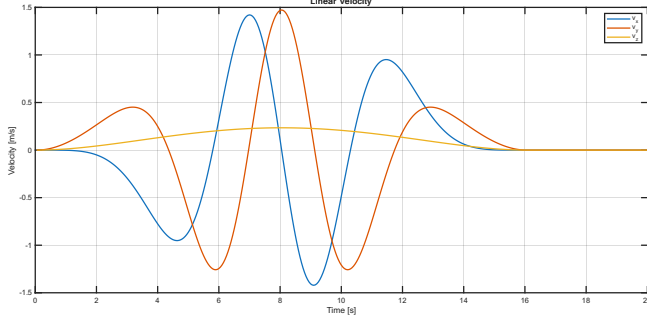


Fig. 2. Desired linear velocity profiles.

Parameter	Nominal value	Perturbed value
m	0.716 kg	1.376 kg
J_x	0.007 kg*m ²	0.014 kg*m ²
J_y	0.007 kg*m ²	0.014 kg*m ²
J_z	0.012 kg*m ²	0.024 kg*m ²
l_r	0.170 m	-
l_l	0.170 m	0.128 m
c_τ	0.160	-

Table 1. Nominal and perturbed values for the quadrotor.

Parameter	Value
p_0	$[0 \ 0 \ 0]^T \ m$
q_0	$[1 \ 0 \ 0 \ 0]^T$
v_0	$[0 \ 0 \ 0]^T \ m/s$
ω_0	$[0 \ 0 \ 0]^T \ rad/s$
T_{spiral}	16 s
$T_{hovering}$	4 s
R_{spiral}	1 m
H_{spiral}	2 m
n_{turns}	2

Table 2. Trajectory values.

dynamics that characterize their evolution. Once the hovering phase is reached, the error decreases rapidly until it reaches a zero value at steady state.

The control inputs shown in Fig. 4 are also characterized by an oscillatory initial phase due to adaptation, while their

Parameter	Value
R	$10I_4$
Q	$diag(4.0e3, 4.0e3, 8.0e4, 1, \dots, 1)$
u_{max}	$2.077e4 \cdot \mathbf{1} \ N$
$T_{horizon}$	3 s
N	60

Table 3. MPC values.

Parameter	Value
ω_{co}	15 rad/s
A_s	$12.8I_6$
B_d	$[I_2 \ 0_{11 \times 2}]^T$
C_d	$[I_2 \ 0_{11 \times 2}]^T$
L_x	$0.5I_{13}$
L_d	$0.5[I_2 \ 0_{2 \times 11}]$
d	$[1 \ 1]^T \ m/s \ height$

Table 4. $\mathcal{L}_1$ and observer MPC values.

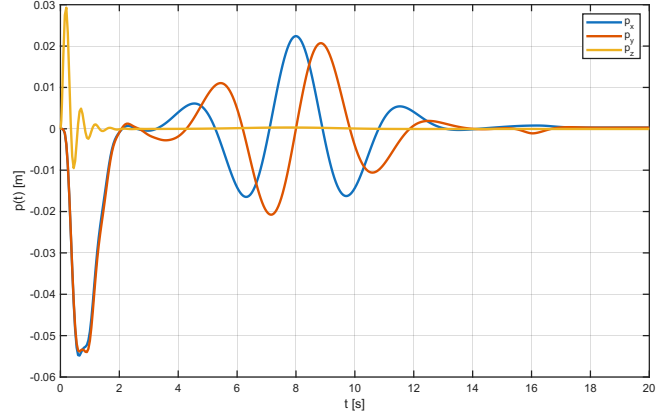


Fig. 3. Position error with adaptive-MPC (model uncertainties).

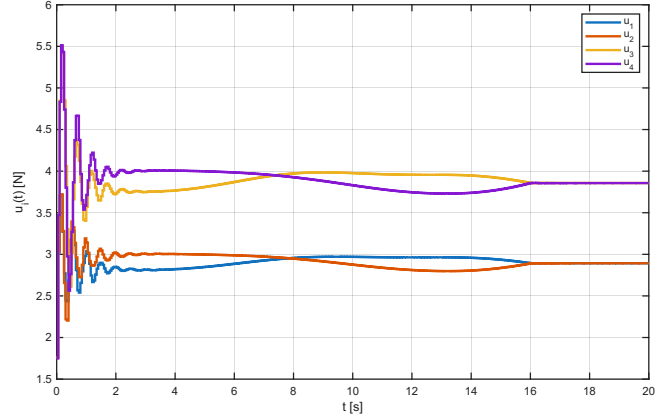


Fig. 4. Control input with adaptive-MPC (model uncertainties).

evolution becomes smooth after approximately two seconds and converge at steady state.

We then test performance using the nominal parameters, but adding the disturbance described above.

Applying the disturbance to this controller reveals its inability to reject it. In fact, we can see a constant offset in the error on the variables affected by the disturbance, which prevents it from converging to zero. This behavior is also reflected in the control inputs, which are unable to compensate for the

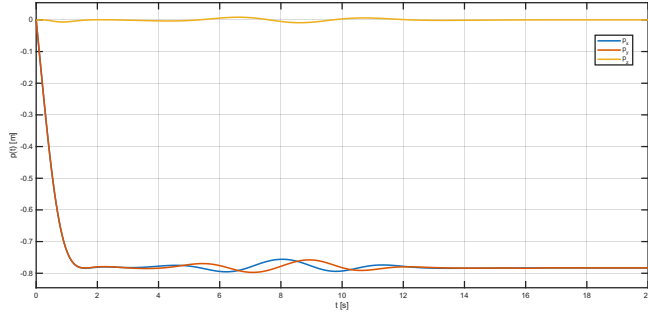


Fig. 5. Position error with adaptive-MPC (additive disturbance).

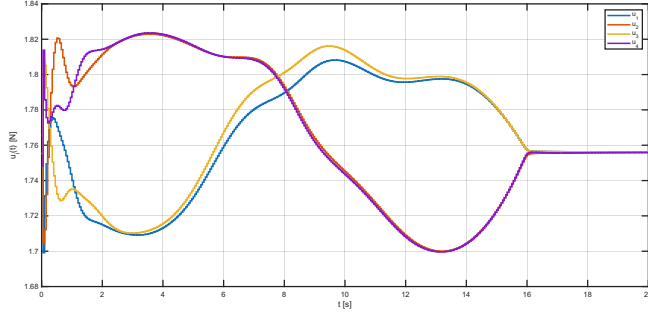


Fig. 6. Control input with adaptive-MPC (additive disturbance).

mismatch (Fig. 5-6).

B. Observer MPC results. Now we test the other controller's performance in the presence of model uncertainties, using the perturbed parameters instead of the nominal ones.

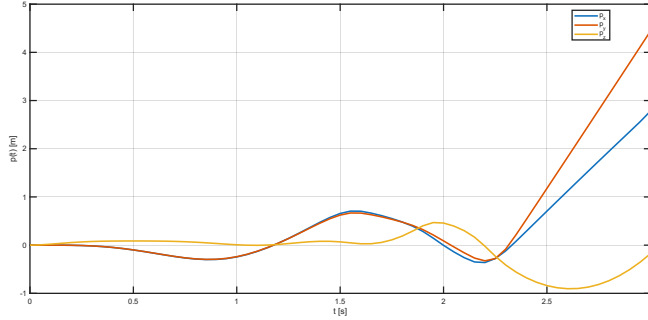


Fig. 7. Position error with observer-MPC (model uncertainties).

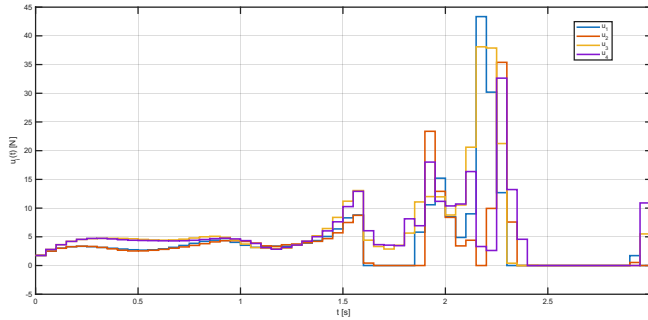


Fig. 8. Control input with observer-MPC (model uncertainties).

Fig. 7-8 show the observer's inability to estimate the uncertainties of the model: both errors and control inputs diverge rapidly, forcing us to stop the simulation much earlier than expected.

We then test performance using the nominal parameters, but adding the disturbance as done before.

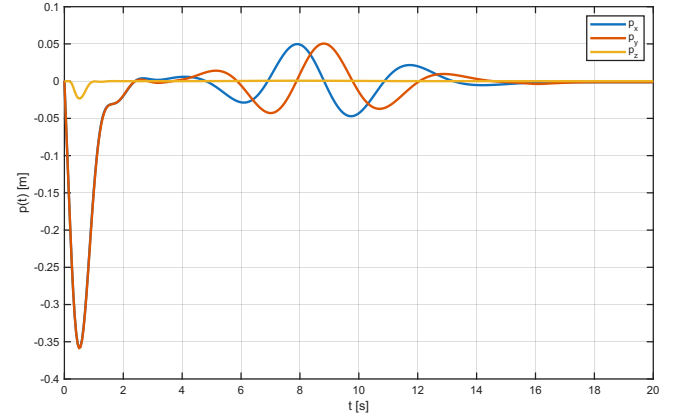


Fig. 9. Position error with observer-MPC (additive disturbance).

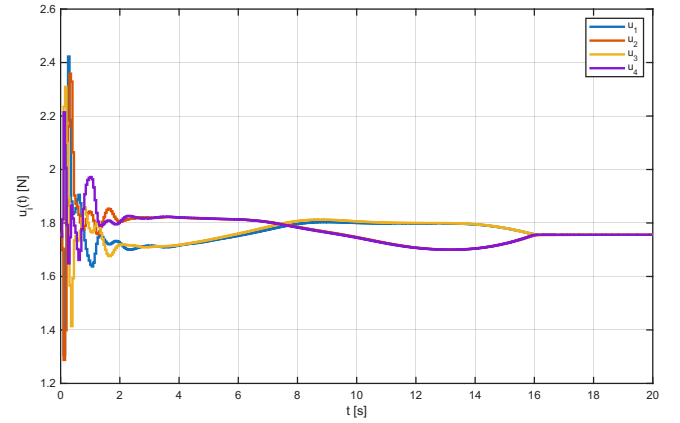


Fig. 10. Control input with observer-MPC (additive disturbance).

In this case, as it can be seen in Fig.9, the error shows little oscillatory behavior, but a fairly high maximum magnitude in the first few moments of the simulation, which, however, decreases rapidly, then evolves smoothly and goes to zero at steady state.

As regards the control inputs, in this case (Fig. 10), in the first few seconds, we can observe oscillations at a very high frequency, which, however, disappear after a few seconds, resuming a good behavior.

Conclusions

As mentioned in the introduction, controlling a UAV in a real environment is a complex field and there is no definitive solution. For this reason, this study sought to understand which controller performs best under a series of working conditions which, although simulated, were designed to be as close as possible to real conditions. The answer was that, as is often the case in complex applications, there is no single solution that is absolutely better than the others. This does not mean that this research was in vain, but simply that the best choice depends on the specific application. For example, we have found that the observer term controller works well even when constant disturbances are present, but it is not able to

compensate for uncertainties in the model: it is preferable in situations where the model is known with a low margin of error, but there is a possibility that the drone will be subject to gusts of wind (inspection of outdoor plants). On the other hand, the controller with adaptation term will work well even when the model is known with a high degree of uncertainty, but it cannot be used successfully if it is subject to external disturbances.

Further developments

One idea that this work gives us concerns merging these two improvements to the standard NMPC that we have analyzed individually here to see if we can get the best out of both techniques: disturbance rejection and robustness to model uncertainties. Although this idea may be interesting and valid, there are at least two important limitations to bear in mind. The first is that complicating the controller also increases the degrees of freedom that need to be managed, which can result in a much longer and more complex tuning procedure than what we saw in the case analyzed. The second concerns computational complexity: we know that MPC in general is very computationally demanding, which was also confirmed during our simulations, so complicating its structure can lead to long computation times that may not be manageable in real time.

Bibliography

1. Sihao Sun Elia Kaufmann Davide Scaramuzza Drew Hanover, Philipp Foehn. Performance, Precision, and Payloads: Adaptive Nonlinear MPC for Quadrotors. *IEEE ROBOTICS AND AUTOMATION LETTERS*, pages 1–8, 2022.
2. Erik Cheever. Fourth Order Runge-Kutta.
3. Luigi Gilermo. Identificazione e Controllo Ottimo, slides of the course. 2024.
4. Ascending Technologies GmbH. AscTec Hummingbird with AutoPilot user's manual. pages 1–28.